



Setting Up Full Multigrid

Louis Moresi¹ 

¹Australian National University

Keywords Underworld Code, Tricks of the Trade, development

A Stokes solve is where the time goes, and multigrid is how we stop it going there. The idea is old and well described: solve the problem on a sequence of grids, let each grid remove the part of the error it can see cheaply, and the work per unknown stops growing with the size of the problem.

What is much less well described is what to do when the hierarchy you have is not the tidy one in the textbook. Ours rarely is. A mesh gets refined where a fault or a boundary layer needs it, so the levels differ by a factor that is not two. A mesh gets cut, or moved, or adapted between timesteps, so the levels were not all built from one another. This note is about the parts that decide whether multigrid actually helps on a mesh like that, and none of them are the options you type into PETSc.

1. TURNING IT ON

Build the mesh with a refinement hierarchy and it is already on:

```
mesh = uw.meshing.UnstructuredSimplexBox(cellSize=0.05, refinement=2)
stokes = uw.systems.Stokes(mesh)
stokes.solve()
```

`refinement=2` builds the mesh twice-refined from a coarse base and keeps the coarse levels. The solver's `preconditioner` property defaults to "auto", which uses geometric full multigrid when the mesh carries a genuine hierarchy — `len(mesh.dm_hierarchy) > 1` — and algebraic multigrid (GAMG) when it does not. You can ask for either explicitly:

```
stokes.preconditioner = "fmg" # geometric; warns and falls back if no hierarchy
stokes.preconditioner = "gamg" # algebraic
stokes.preconditioner_settings # what actually got applied
```

`preconditioner_settings` is worth knowing about. It reports the option keys UW3 has put in the database for this solver, so what was applied can be asserted on rather than inferred from how long the solve took.

Two things about "auto" that are deliberate. It only ever *adds* geometric multigrid to an untouched default: if you have configured the preconditioner yourself, it leaves your configuration alone. And on a single-field scalar or vector solver it stays with GAMG even when a hierarchy exists, because the native geometric path there is unreliable; asking for "fmg" explicitly routes it through custom transfers instead. Where a request is declined, the reason is recorded in `pc_fallbacks` rather than being silent.

2. A LEVEL IS A COARSENING RATIO

This is the one that costs the most time when it is wrong.

A multigrid level earns its place by seeing error that no other level can see cheaply. That is a statement about *resolution*: each level should differ from its neighbour by something near a factor of two in cell size. It is not a statement about how the mesh was made.

Refinement engines do not work that way. A pass is how an engine *reaches* a target size — independence conditions cap how many edges may be split at once, and a conforming closure cascades — so the number of passes reflects the engine's constraints, not the geometry. Recording one level per pass produces levels that coarsen almost nothing. We measured ratios of 1.06, 1.02 and 1.007 sitting in a hierarchy, each costing a full Galerkin triple product and a smoother sweep on every cycle to remove error that its neighbour had already removed.

Cutting a mesh has the same problem in a purer form. A cut re-represents the same grid with a surface conformed to it; it adds no resolution at all. Two cuts once produced two levels coarsening by 1.11 and 1.17 against a threshold of 1.8.

The cost is not subtle. On a box-fault shear solve, dropping the levels that were not genuine coarsenings:

levels	how	wall clock
9	one per pass	93.7 s
7	mg_coarsening_ratio=2.0	47.5 s
5	mg_coarsening_ratio=3.0	23.3 s

Same mesh, same answer, four times the speed.

Measure resolution, not cell count

Under adapt-on-top the mesh only grows where the feature is, so a genuine halving of h can show as a global cell-count ratio near 1. Counting cells to decide whether a level is real will tell you that every level is redundant on exactly the meshes where the levels matter most.

3. LEVELS THAT WERE NOT BUILT FROM EACH OTHER

The textbook hierarchy is nested: every coarse node is a fine node, every coarse cell is a union of fine cells, and the transfer between them follows from the refinement relation. When that holds you can write the prolongation down exactly, for any element degree, from the parent-cell map alone. With the Lagrange basis dual to its nodal points,

$$W = B M^{-1}, \quad M_{im} = \mu_m(x_i), \quad B_{tm} = \mu_m(x_t), \quad (1)$$

where μ_m are the parent cell's basis functions, x_i its own nodes and x_t the fine degrees of freedom inside it. No reference element, no per-degree formulae, no point location. One detail is not optional: pull the coordinates back through the parent's own vertices before evaluating. Raw monomials give a conditioning that grows like h^{-k} — around 10^6 at P_3 — while under the pullback the conditioning depends only on the degree and the dimension.

Often, though, the levels were *not* built from each other. A mesh that has been cut, moved or adapted has coarse levels that are related to the fine one geometrically but not combinatorially. Then the transfer has to be constructed rather than derived, and UW3 falls back to locating fine degrees of freedom inside a triangulation of the coarse degree-of-freedom cloud.

That fallback works, and it is worth being clear about *why* it works, because it is not the reason one might assume. The triangulation is of the coarse **degree-of-freedom points**, and it never consults the mesh cells. How often its simplices agree with the actual cells:

mesh	agreement with real cells
2D uniform, P_1	100%
2D adapt child	90.8%
3D uniform, P_1	58.8%
3D adapt child	17.1%

At 17% agreement the construction is not reproducing the mesh in any meaningful sense. It works because it reproduces *linear functions* exactly, and that is the property multigrid transfers actually need. Knowing this changes what you check when a hierarchy underperforms in 3D: the question is not whether the transfer found the right cells, because mostly it did not.

4. THE OUTER SOLVER HAS TO BE FLEXIBLE

This one is not about the hierarchy at all, and it is the single largest effect we have measured on a hard nonlinear problem.

In the Schur factorisation used for Stokes, the velocity block is solved with its own Krylov method. That solve is not a passive preconditioner: it computes the search direction. Being Krylov, it is inexact, and being inexact, the operator effectively applied *differs between outer iterations*. A non-flexible GMRES assumes a fixed preconditioner and its residual recurrence has no guarantee against an operator that drifts.

On a notch problem at refinement 3, varying only this:

outer	inner rtol	velocity its/step	result
gmres	3.3e-08	983	DIVERGED_LINEAR_SOLVE
gmres	1e-03	915	DIVERGED_LINEAR_SOLVE
fgmres	3.3e-08	58	no linear failure
fgmres	1e-03	23	no linear failure

Raising the velocity iteration cap from 200 to 2000 changed nothing at all — the rows were byte-identical. Loosening the inner tolerance by five orders of magnitude while the outer stayed non-flexible was worth 7%. The flexible outer Krylov is the entire effect, and once it is in place the same loosening is worth a further factor of 2.6.

The inner solves being deliberately inexact is an old design, inherited from Citcom. Two halves of one decision: inexact inner solves require a flexible outer method, and *how* inexact is bounded by requiring the inner solves to converge well below the tolerance demanded of the outer one. The factors UW3 uses — 0.033 for velocity, 0.1 for pressure — are that margin. Their existence is principled; their particular sizes are inherited convention.

5. WHAT IT IS WORTH

The comparison that matters is against algebraic multigrid, which is what you get without a hierarchy and which needs nothing from the mesh.

On a refined mesh with a genuine hierarchy, geometric multigrid is substantially better conditioned, because it is built from the refinement relation rather than inferred from the operator's connectivity — which is exactly what mesh anisotropy makes misleading. In the runs behind the [mesh-mover note](#), a four-level pc=mg converged the velocity block in three to four iterations; where the hierarchy was lost and the solve fell back to algebraic multigrid, the same block took 23.

The corollary is the reason those two notes are a pair. A hierarchy is a capital asset: it is built once and it is what makes the solve cheap. Anything that destroys it — remeshing in particular — is not only paying for the remesh, it is paying for every solve afterwards until a new hierarchy is built. That is what moving the mesh instead of remaking it is protecting.

6. USING IT

```
# A hierarchy: two refinements from a coarse base, coarse levels kept.
mesh = uw.meshing.UnstructuredSimplexBox(cellSize=0.05, refinement=2)

stokes = uw.systems.Stokes(mesh)
stokes.preconditioner = "auto"          # the default: FMG if there is a hierarchy
stokes.solve()

print(stokes.preconditioner_settings) # what was applied
print(stokes.pc_fallbacks)           # anything declined, and why
```

When adapting, ask for levels by coarsening ratio rather than taking one per engine pass:

```
child = mesh.adapt(metric, max_levels=3, mg_coarsening_ratio=2.0)
```